Explainable AI
White-Box Methods



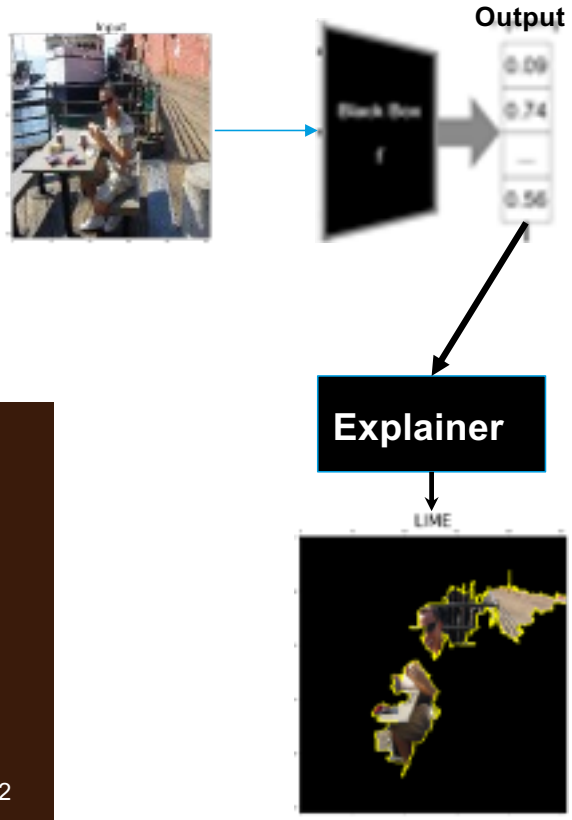
Pr. Jenny Benois-Pineau



1. Introduction

CNNs : From Black-Box to White-Box Explanations

BLACK-BOX METHODS



Explanation for the given input by the network

WHITE-BOX METHODS

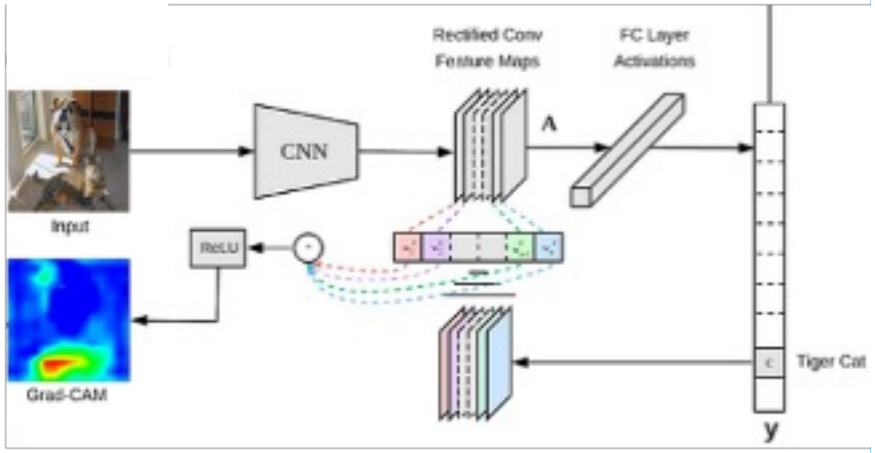


Fig: Grad-CAM method [1]

These methods utilize the structure, activations or information from the network to produce the explanations for a given input image.

[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Explanation Methods

TAXONOMY of Methods [1] Analyses of trends: Based on 12 most popular works

Black-Box Based

Occlusions – Zeiler'13

LIME – Ribeiro'16

Meaningful Perturbations – Fong'17

Approximation of DeepNN classifiers- White-Box

Vanilla Backpropagation – Simonyan'14

DecovNet - Zeiler'14
Smooth Grad - Smilkov'16

Gradient x Input – Shrikumar'16

Structure Based - White-box

LRP – Bach'15, DTD - Montovan'17

Grad-CAM – Selvaraju'16

DeepLIFT – Shrikumar'17

FEM - Fuad'2020 (LaBRI)

Adversarial Based

Explanations on adversarial examples - Ignatiev'19

Similarity & Adversarial learning - Charachon'21

[1] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901..

Methods based on Linearization of the Deep-CNN

CNN is a non-linear classifier that is mapping $f(x)$ from the input space to the output score space S

These Explanation methods approximate the non-linear $f(x)$ as:

$$S_c(x) \approx w^T x + b$$

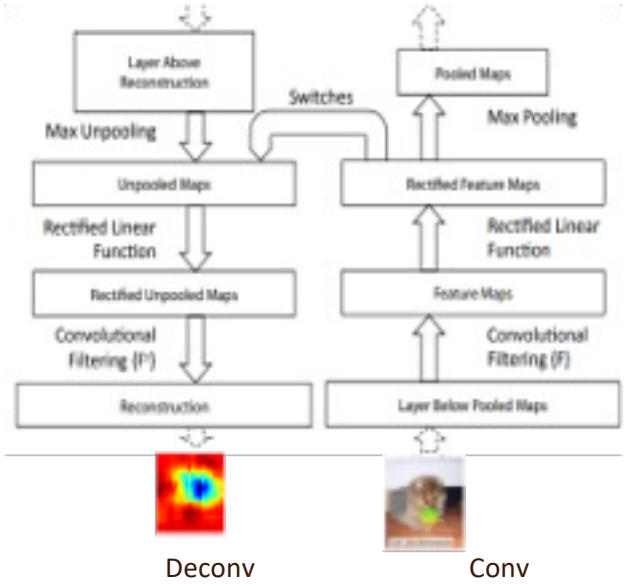
where w are the weights, x is the input and b is the bias of the network.

Using this approximation these methods try to find the weights that relate the output score S to the particular score: i.e. they try to find the gradient of the score wrt to the input x such that the weights of the inputs that contributed the most to the particular output score can be computed.

DeconvNet [1]

- The Deconvolution Network works by reversing the mapping of a CNN. It begins at the output score of the classifier space and maps it to the input pixels of a given image.

- It uses the structure of the CNN and the learned filters of the CNN layers to do this mapping.



- As seen in the image - the deconv layers are attached to the corresponding CNN conv layers.

- First an input image is passed through the CNN and the output is computed.

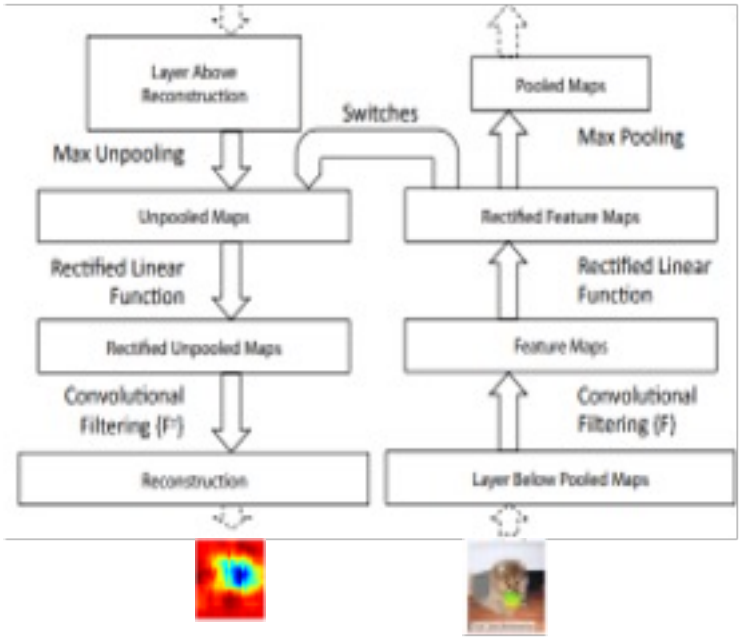
- We examine the activations of the network for a particular class **c** (either the predicted class or the ground truth label class - to see what pixels in the input image contribute the most to the output score of that class **c**)

Therefore, in the output layer only the activation related to this class **c** is considered and all else is set to 0.

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.
[2] Zeiler, M.D., Taylor, G.W. and Fergus, R., 2011, November. Adaptive deconvolutional networks for mid and high level feature learning. In *2011 ICCV* (pp. 2018-2025).

DeconvNet [1]

- Beginning at the output layer - propagate the through each of the Deconv layers.



- To do this calculation the Deconv layers reverse the 3 major operations of the CNN layers.

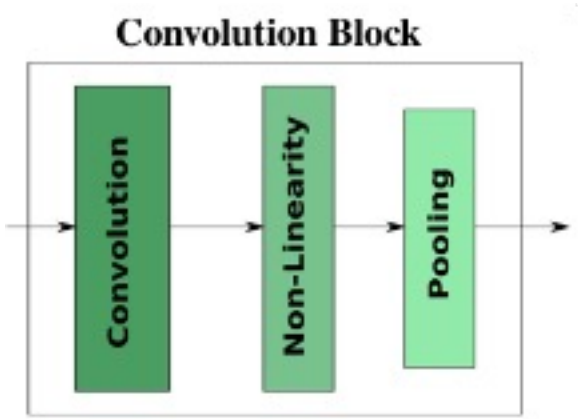


Fig: A typical convolution block [2]

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

DeconvNet [1]

The 3 stages of that the DeconvNet uses are:

- Pooling → **Unpooling**
- Non-linearity → **Rectification**
- Convolution → **Filtering**

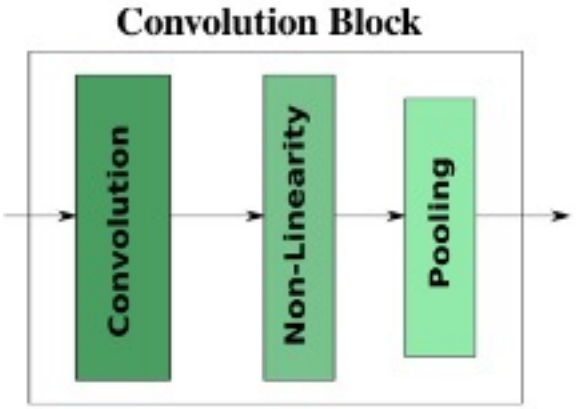


Fig: A typical convolution block [2]

- NOTE: No further training is required for the DeconvNet

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

DeconvNet [1]

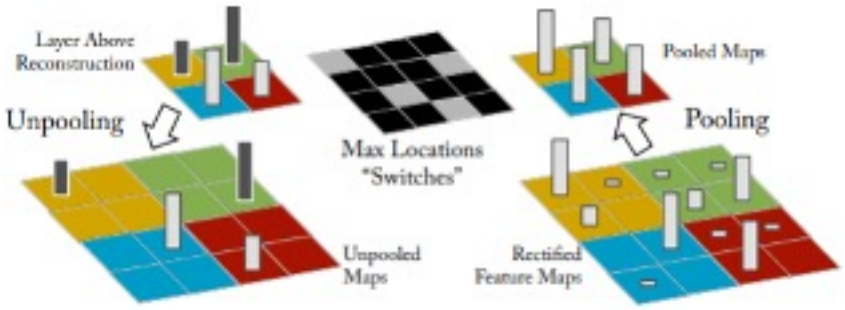


Fig 1: Image taken from [1]

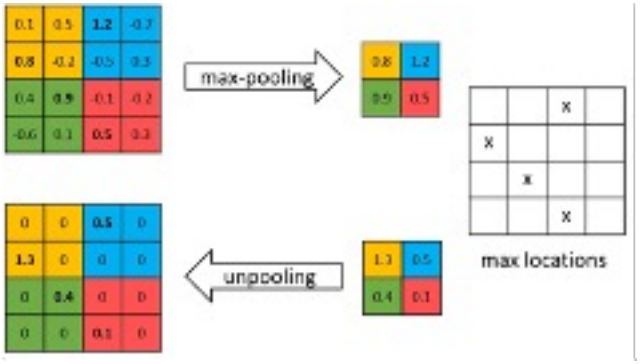


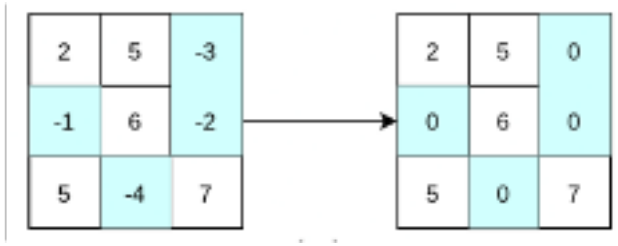
Fig 2: Max Unpooling operation

Step 1: UNPOOLING

- CNNs typically use Max Pooling which is non-invertible.
- Thus, during the forward pass of the Conv Layers the method saves **switch variables** - stores the location of the maxima for each pooling region
- During unpooling, these switch locations are used as shown in Fig 2

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

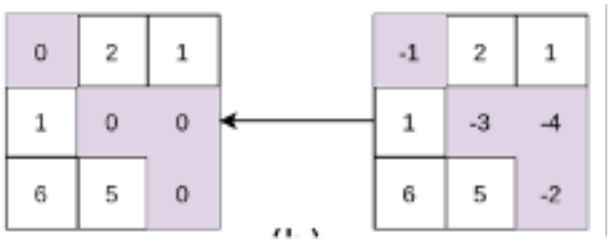
DeconvNet [1]



ReLU during Forward Pass

Step 2: RECTIFICATION

- Applies ReLU non-linearity to the unpooled maps
- Propagate only the non-negative values.



ReLU during rectification

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]

Step 3: FILTERING

Inversion of Convolution – Convolution of unpooled rectified map with 180 (degrees) inverted **filter**

The filter learnt by the CNN is transposed and applied to the Rectified Unpooled Maps

a	b	c
d	e	f
g	h	i

Filter during Convolution

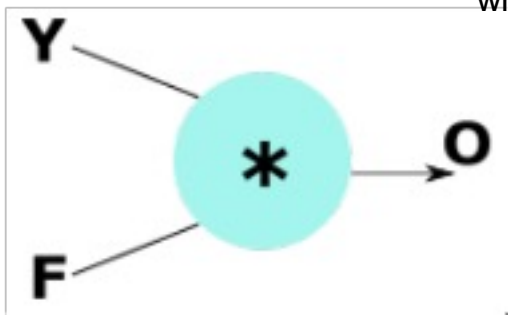
i	h	g
f	e	d
c	b	a

Filter during Deconvolution

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]

For a convolution neuron shown in the figure the equation would be: $Y * F = O$



where Y: input, F: conv layer filter,O: output

During Backpropagation of loss L:

Input: $\frac{\partial L}{\partial O}$ the gradients calculated are: $\frac{\partial L}{\partial Y}$ and $\frac{\partial L}{\partial F}$

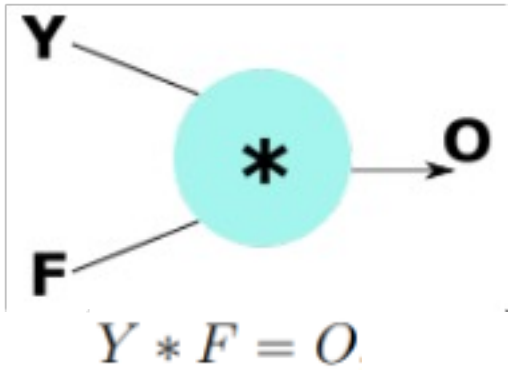
Using Chain rule: $\frac{\partial L}{\partial Y} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial Y}$

For a detailed calculation let Y be 3 X 3 matrix, F is 2 X 2 and convolution operation stride = 1

$$Y = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ y_{31} & y_{32} & y_{33} \end{bmatrix}, F = \begin{bmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \end{bmatrix}, O = \begin{bmatrix} o_{11} & o_{12} \\ o_{21} & o_{22} \end{bmatrix}$$

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]



$$Y = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ y_{31} & y_{32} & y_{33} \end{bmatrix}, F = \begin{bmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \end{bmatrix}, O = \begin{bmatrix} o_{11} & o_{12} \\ o_{21} & o_{22} \end{bmatrix}$$

FORWARD PASS:

$$o_{11} = y_{11}f_{11} + y_{12}f_{12} + y_{21}f_{21} + y_{22}f_{22}$$

$$o_{12} = y_{12}f_{11} + y_{13}f_{12} + y_{22}f_{21} + y_{23}f_{22}$$

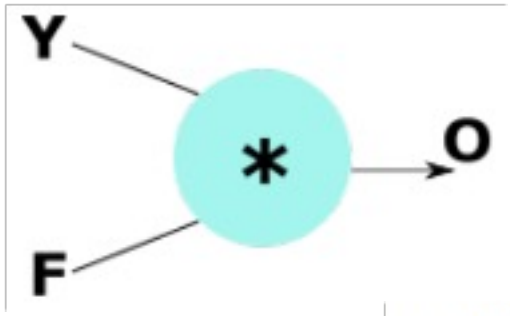
$$o_{21} = y_{21}f_{11} + y_{22}f_{12} + y_{31}f_{21} + y_{32}f_{22}$$

$$o_{22} = y_{22}f_{11} + y_{23}f_{12} + y_{32}f_{21} + y_{33}f_{22}$$

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]

This will be the input to the preceding layer during backpropagation $\frac{\partial L}{\partial Y} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial Y}$



$$Y * F = O$$

$$Y = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ y_{31} & y_{32} & y_{33} \end{bmatrix}, F = \begin{bmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \end{bmatrix}, O = \begin{bmatrix} o_{11} & o_{12} \\ o_{21} & o_{22} \end{bmatrix}, \frac{\partial L}{\partial O} = \begin{bmatrix} \frac{\partial L}{\partial o_{11}} & \frac{\partial L}{\partial o_{12}} \\ \frac{\partial L}{\partial o_{21}} & \frac{\partial L}{\partial o_{22}} \end{bmatrix}$$

FORWARD PASS

$$o_{11} = y_{11}f_{11} + y_{12}f_{12} + y_{21}f_{21} + y_{22}f_{22}$$

$$o_{12} = y_{12}f_{11} + y_{13}f_{12} + y_{22}f_{21} + y_{23}f_{22}$$

$$o_{21} = y_{21}f_{11} + y_{22}f_{12} + y_{31}f_{21} + y_{32}f_{22}$$

$$o_{22} = y_{22}f_{11} + y_{23}f_{12} + y_{32}f_{21} + y_{33}f_{22}$$

For calculating $\frac{\partial O}{\partial Y}$

$$\frac{\partial o_{11}}{\partial y_{11}} = f_{11}, \frac{\partial o_{11}}{\partial y_{12}} = f_{12}, \frac{\partial o_{11}}{\partial y_{21}} = f_{21}, \frac{\partial o_{11}}{\partial y_{22}} = f_{22}$$

And so on for all the terms

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]

This will be the input to the preceding layer during backpropagation $\frac{\partial L}{\partial Y} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial Y}$

Thus partial gradient of the loss L w.r.t input Y will be

$$\frac{\partial L}{\partial y_{11}} = \frac{\partial L}{\partial o_{11}} \cdot f_{11}, \quad \frac{\partial L}{\partial y_{12}} = \frac{\partial L}{\partial o_{11}} \cdot f_{12} + \frac{\partial L}{\partial o_{12}} \cdot f_{11}, \quad \frac{\partial L}{\partial y_{13}} = \frac{\partial L}{\partial o_{12}} \cdot f_{12}$$

$$\frac{\partial L}{\partial y_{21}} = \frac{\partial L}{\partial o_{11}} \cdot f_{21} + \frac{\partial L}{\partial o_{21}} \cdot f_{11}, \quad \frac{\partial L}{\partial y_{22}} = \frac{\partial L}{\partial o_{11}} \cdot f_{22} + \frac{\partial L}{\partial o_{12}} \cdot f_{21} + \frac{\partial L}{\partial o_{21}} \cdot f_{12} + \frac{\partial L}{\partial o_{22}} \cdot f_{11}$$

$$\frac{\partial L}{\partial y_{23}} = \frac{\partial L}{\partial o_{12}} \cdot f_{22} + \frac{\partial L}{\partial o_{22}} \cdot f_{12}, \quad \frac{\partial L}{\partial y_{31}} = \frac{\partial L}{\partial o_{21}} \cdot f_{21}$$

$$\frac{\partial L}{\partial y_{32}} = \frac{\partial L}{\partial o_{21}} \cdot f_{22} + \frac{\partial L}{\partial o_{22}} \cdot f_{21}, \quad \frac{\partial L}{\partial y_{33}} = \frac{\partial L}{\partial o_{22}} \cdot f_{22}$$

This can be written as a full convolution of the 180 inverted convolution filter matrix: flipped vertically and then horizontally

$$F = \begin{bmatrix} f_{22} & f_{21} \\ f_{12} & f_{11} \end{bmatrix}$$

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]

Step 3: FILTERING

Inversion of Convolution – Convolution of unpooled rectified map with 180 (degrees) inverted **filter**

The filter learnt by the CNN is transposed and applied to the Rectified Unpooled Maps

a	b	c
d	e	f
g	h	i

Filter during Convolution

i	h	g
f	e	d
c	b	a

Filter during Deconvolution

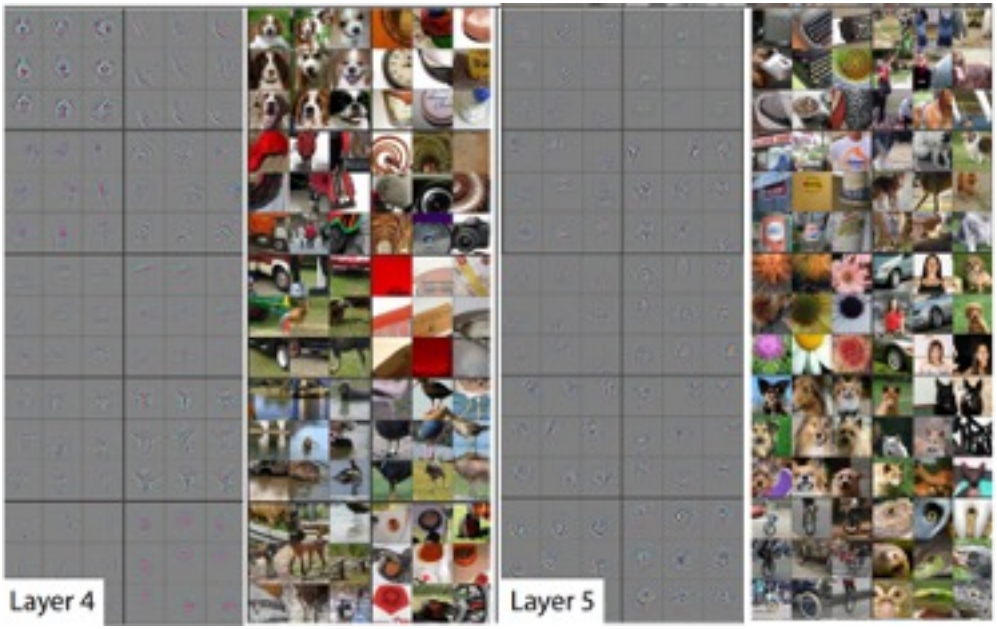
Thus this filtering step is equivalent to

The calculation of gradient wrt to the input for a convolution layer.

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

DeconvNet [1]





Visualization of features in a fully trained model from different layers. The images show the top 9 activations in a random subset
of feature maps across the validation data, projected down to pixel space using the deconvolutional network approach. Results presented from [1]

[1] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

Vanilla Backpropagation [1]

$$S_c(x) \approx w^T x + b$$

- Similar to DeconvNet, the method attempts to find the pixels in a particular input image that contributed to the output of a particular class by the network.
- As shown in Fig 1, top row: input image and bottom row: relevance score map with a **value per pixel.**
- Uses simple backpropagation of the output score wrt to the Input

$$R_{ij} = \left\| \frac{\partial}{\partial x_{ij}} S_c \right\|$$

R: relevance score
S: output score for the class c
X: particular input image

i,j: the height and width position for a particular pixel in the 2D image x



Fig 1: Results of the Gradient Backpropagation method [1]

[1] Simonyan, K., Vedaldi, A. and Zisserman, A., 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034.

Vanilla Backpropagation [1]

- During training of a CNN, the backpropagation step stops at the second layer of the network as we do not change the input values.

- But, with this method the backpropagation is performed till the input layer to inspect which pixels affect the output the most.

- For an RGB image the weight (w)/scores of each pixel for all the 3 channels are computed at the input and the final relevance map is computed as:

$$R_{ij} = \max_k |w(i,j,k)|$$

here *k:* refers to the channels

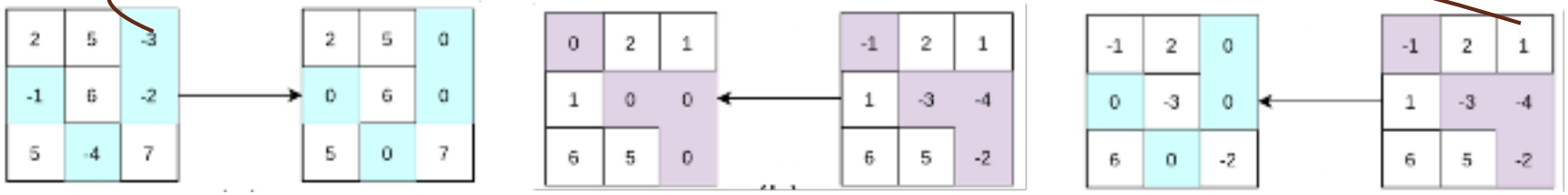


Fig 1: Results of the Gradient Backpropagation method [1]

[1] Simonyan, K., Vedaldi, A. and Zisserman, A., 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034.

Vanilla Backpropagation [1] vs DeconvNet [2]

- Vanilla Backpropagation is the **generalization** of DeconvNet
- Comparing the three operations of a DeconvNet
  - **Unpooling**: Typical backpropagation uses the **Max location switches** matrix like DeconvNet **Same**
  - **Rectification:** With Vanilla Backprop the ReLU is applied based on the feature map of the input to the layer in the forward pass. The reason is : if the features were negative in the forward pass, then we do not need to take them into account in the backward explanation. **Different**



ReLU during Forward Pass | ReLU: DeconvNet | ReLU: Backpropagation

  - **Filtering:** As shown in the previous calculation: the vertically and horizontally flipped conv filter used for deconvolution is the same as calculating the gradient of the convolution wrt to the input *x*
    Same

[1] Simonyan, K., Vedaldi, A. and Zisserman, A., 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034.
[2]Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. ECCV (pp. 818-833). Springer, Cham.

Guided Backpropagation [1]

- With the Vanilla Backpropagation, some **negative gradients** might still be propagated and would decrease the activations in the layers we want to visualize

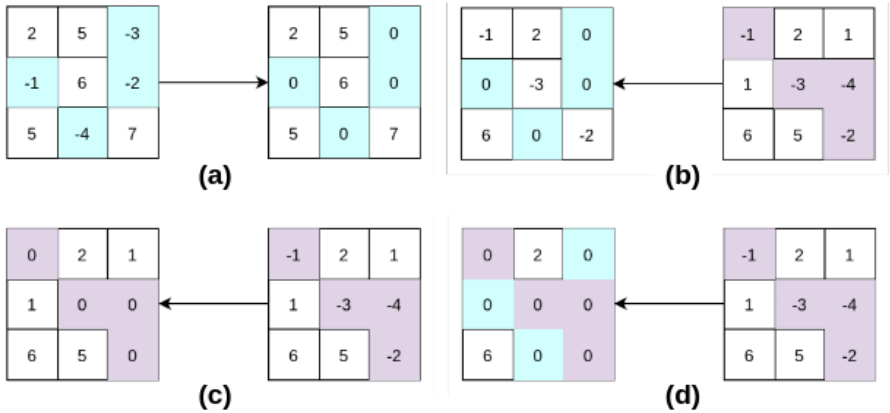


Fig 6 The ReLU operation during (a) Forward Pass (b) Backpropagation (c) Backpropagation with DeconvNet (d) Guided Backpropagation

- Thus, a ***guidance*** is used where the negative values during the forward pass and also after the unpooling are nullified during the backpropagation step.

- This combines the ReLU used by the DeconvNet and the Vanilla Backpropagation method.

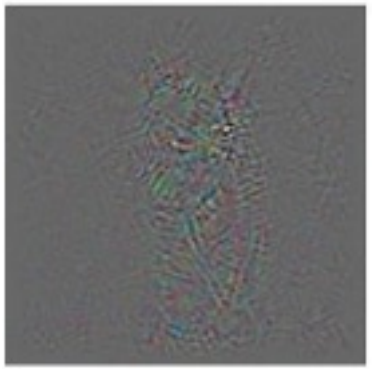
[1] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, Martin A. Riedmiller: Striving for Simplicity: The All Convolutional Net. ICLR (Workshop) 2015
[2] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. In *ECCV* (pp. 818-833). Springer, Cham.

Guided Backpropagation [1]

- It results in sharper visualizations for the descriptive regions in the input image.



Actual Image



Vanilla Backpropagation



Guided Backpropagation

[1] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, Martin A. Riedmiller: Striving for Simplicity: The All Convolutional Net. ICLR (Workshop) 2015
[2] Zeiler, M.D. and Fergus, R., 2014, September. Visualizing and understanding convolutional networks. In *ECCV* (pp. 818-833). Springer, Cham.

Integrated Gradients (IG) [1]

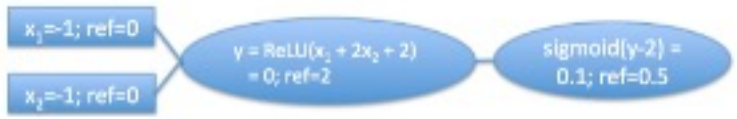


Figure 1. Simple network with inputs x_1 and x_2 that have reference values of 0. When $x_1 = x_2 = -1$, output is 0.1 but the gradients w.r.t x_1 and x_2 are 0 due to inactive ReLU y (which has activation of 2 under reference input). By comparing activations

- The figure shows a simple ReLU based network and in the second stage *y = ReLU(-1) = 0.* The output of the network is 0.1
- When computing the gradient for x1 and x2 the result will be 0 as in the gradient based methods the rectification operation sets the negative values to 0.
- The authors of IG argue that the rectification step of gradient based methods would cause features that may affect the output to be not considered in the explanation at all.
- Thus they suggest to get the explanations as an integration of the gradients along the path from a baseline to the input i.e. to find a path where the feature is absent (0) to the where it is present (1)

[1] Sundararajan, M., Taly, A. and Yan, Q., 2017, July. Axiomatic attribution for deep networks. In International conference on machine learning (pp. 3319-3328). PMLR.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal o
Imaging, 30(5), pp.050901-050901.

Integrated Gradients [1]

- In case of image classification the **baseline** is taken as shown in the first block in the image shown below: the black/empty image.
- Progressively, small steps are taken from 0 (black image pixel) to 1 (the pixel's actual value) : the figure shows the images obtained from the black to final input image taken in 7 steps.

$$x(\alpha) = x' + \alpha \times (x - x')$$

here x' is the baseline black image, x is the input image and α ϵ [0,1] is the parameter that controls the number of steps to transform baseline to input image



Fig 8 Transformation of the baseline image for integrated gradients for 7 values of $\alpha = [0, 1]$. Image has been taken from ImageNet database[18]

Image taken from [2]

[1] Sundararajan, M., Taly, A. and Yan, Q., 2017, July. Axiomatic attribution for deep networks. In International conference on machine learning (pp. 3319-3328). PMLR.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal o
Imaging, 30(5), pp.050901-050901.

Integrated Gradients [1]

- Thus, if we consider x_i the value of the *i*-th feature from *x* then the following equation shows the calculation of the integrated gradients:

$$R_i^c = (x_i - x_i^{'}) \times \int_{\alpha=0}^{1} \frac{\partial S_c(x^{'} + \alpha \times (x - x^{'})}{\partial x_i} d\alpha$$

where R is the relevance score map, the partial derivative is the gradient that is calculated using the vanilla backpropagation method, and integration is approximated by a summation over fixed number of samples.



Fig 8 Transformation of the baseline image for integrated gradients for 7 values of $\alpha = [0, 1]$. Image has been taken from ImageNet database[18]

Image taken from [2]

[1] Sundararajan, M., Taly, A. and Yan, Q., 2017, July. Axiomatic attribution for deep networks. In International conference on machine learning (pp. 3319-3328). PMLR.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal o
Imaging, 30(5), pp.050901-050901.

Integrated Gradients [1]

- In simpler words, the following equation implemented as:

$$R_i^c = (x_i - x_i^{'}) \times \int_{\alpha=0}^{1} \frac{\partial S_c(x^{'} + \alpha \times (x - x^{'})}{\partial x_i} d\alpha$$

1. Calculate the vanilla backpropagation gradients for each of images (eg for the 7 images in Fig)
2. Sum the value of the gradients calculated for each of the image
3. Weight the term by the difference in the initial baseline value of the pixel and value of the pixel in input image
4. Do this for each pixel to get the integrated gradient relevance score map.



Fig 8 Transformation of the baseline image for integrated gradients for 7 values of $\alpha = [0, 1]$. Image has been taken from ImageNet database[18]

Image taken from [2]

[1] Sundararajan, M., Taly, A. and Yan, Q., 2017, July. Axiomatic attribution for deep networks. In International conference on machine learning (pp. 3319-3328). PMLR.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal o
Imaging, 30(5), pp.050901-050901.

Integrated Gradients [1]

- As seen from the image more distinctive features from the input image are visualized (more sharper)

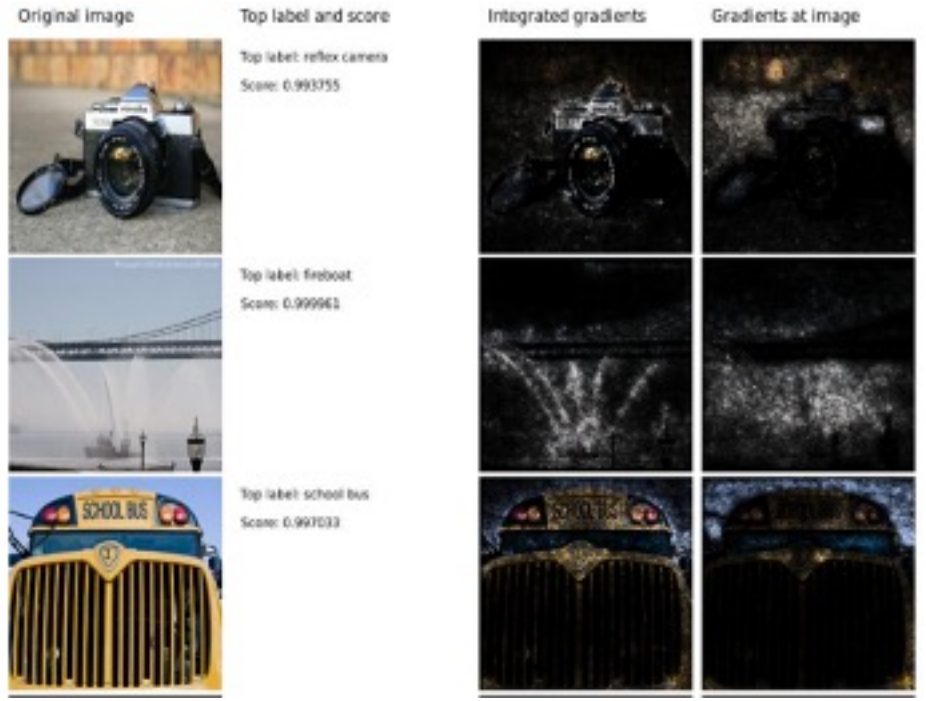


Image taken from [1]

[1] Sundararajan, M., Taly, A. and Yan, Q., 2017, July. Axiomatic attribution for deep networks. In International conference on machine learning (pp. 3319-3328). PMLR.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal o
Imaging, 30(5), pp.050901-050901.

Smooth Grad [1]

- Previous methods had the issue of noisy saliency maps - the authors suggested the idea of getting sharper maps by averaging over multiple noisy maps.
- They add a small Gaussian noise to image and generate *n* such images. x_i in the equation denotes a noisy image created by adding this Gaussian noise.
- They also observe hat adding about **10-20%** of noise to the sampled images produced sharper maps.
- The saliency maps are calculated for each of these *n* noisy images and their average gives the final relevance score map for the image *x*.

$$x_i = x + \mathcal{N}(0, \sigma^2)$$

$$\hat{R}^c(x) = \frac{1}{n} \sum_{i=1}^{n} R^c(x_i)$$

Choice of σ is such that

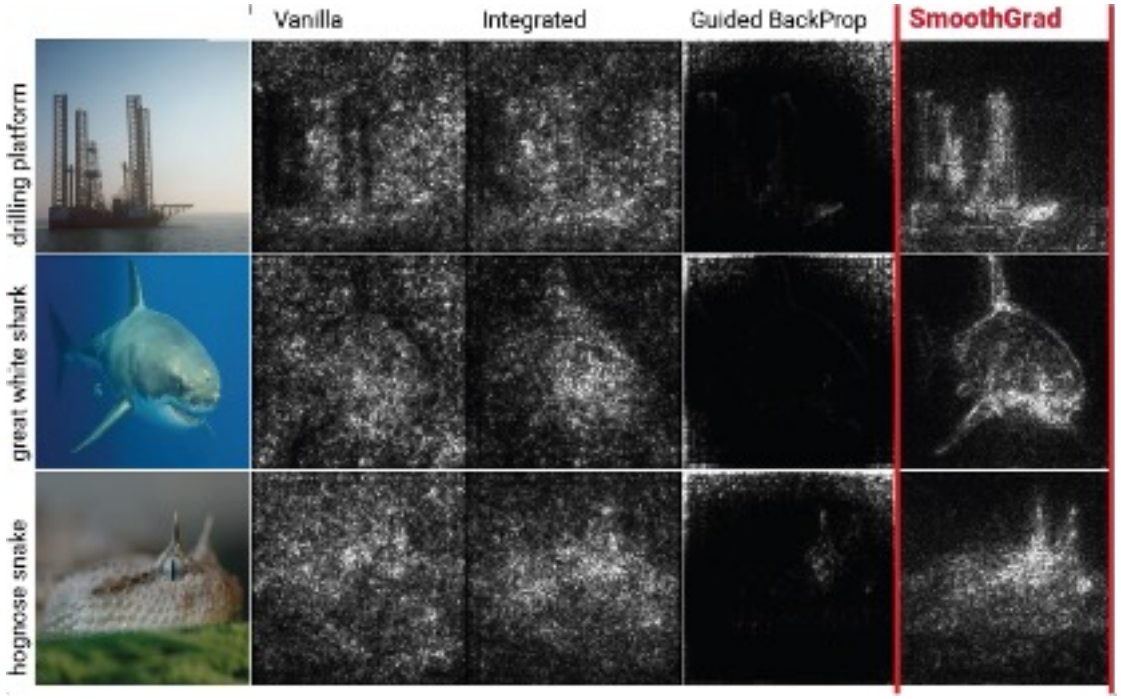
$\frac{\sigma}{x_{max} - x_{min}}$ Is in the range of [0.1,0.2]

where x_{max} and x_{min} are the maximum and minimum values of the pixels of the image

[1] Smilkov, D., Thorat, N., Kim, B., Viégas, F. and Wattenberg, M., 2017. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*.

Smooth Grad [1]

- Smooth grad has better visualizations when compared to the other methods
- Also, it can be combined with any explanation method



Results from [1]

[1] Smilkov, D., Thorat, N., Kim, B., Viégas, F. and Wattenberg, M., 2017. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*.

Grad-CAM [1]

- Gradient-weighted Class Activation Mapping (Grad-CAM) visualizes class discriminative activations of a network for a particular input image
- Deeper Conv layers of a CNN act as high-level feature extractors [2], hence contain better abstract information about the class present in the image
- Instead of propagating all the way to the input - authors suggest to use the output of only the last conv layer of the network.
- Result : A coarse localization map that highlights the important regions in the input image for the prediction of the particular class.

- Method
  - Calculate the gradient of the output score for a class **c** for a an input image **x** at the desired conv layer (usually the last conv layer of the network)
  - Use this to calculate the weights of each activation map in last conv layer using Global Average Pooling of the gradients
  - Multiply each activation map with these weights, perform the weighted sum of the maps and
  - Upscale this map to the size of the input image to get the final heat map.

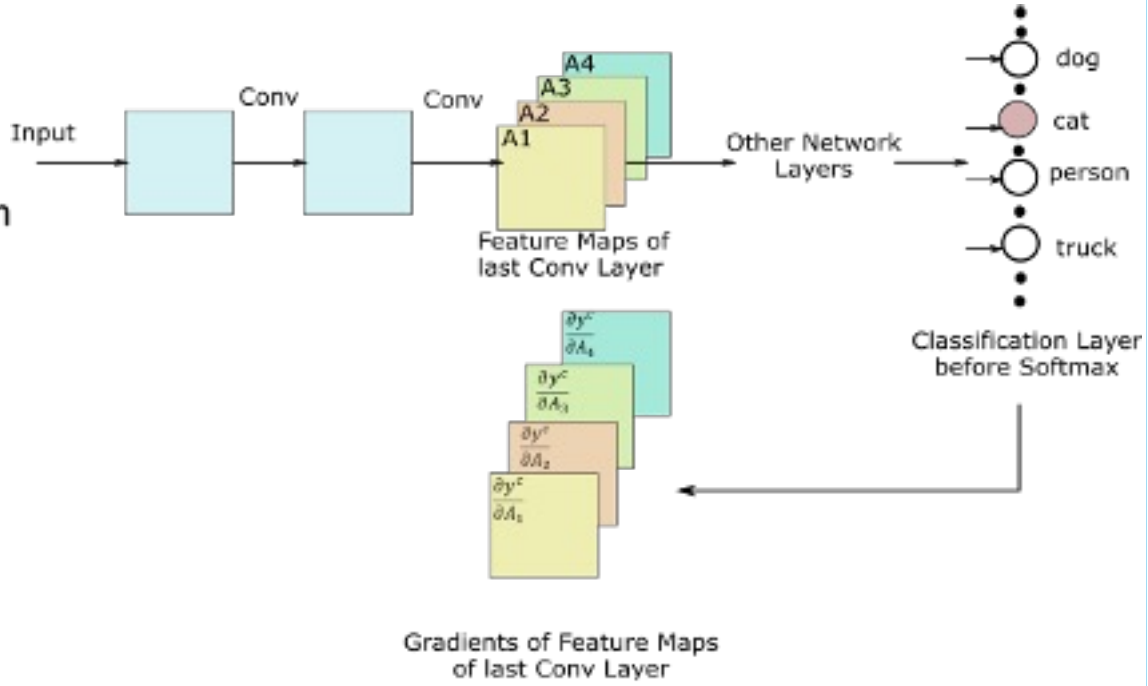
[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).
[2]Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," IEEE Trans. on pattern analysis and machine intelligence 35(8), 1798–1828 (2013)

Grad-CAM [1]

Step 1 : Compute gradients: $\frac{\partial y^c}{\partial A^k}$

- *Exploit Spatial Information* : Conv Layer
- *Gradient* : Transfer information from class decision to spatial features
- Based on Backpropagation

c: class
y^c : Score of the class *c*
k: size of last conv layer
A: activations/ output

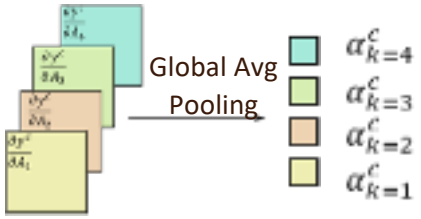


[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Grad-CAM [1]

Step 2 : Calculate Weights by Global Average Pooling of Gradients

$$\alpha_k^c = \overbrace{\frac{1}{Z}\sum_i\sum_j}^{\text{global average pooling}} \underbrace{\frac{\partial y^c}{\partial A_{ij}^k}}_{\text{gradients via backprop}}$$



Step 3 : Final Heat Map : Weighted combination of Activation maps

$$L^c_{\text{Grad-CAM}} = ReLU\underbrace{\left(\sum_k \alpha_k^c A^k\right)}_{\text{linear combination}}$$

1. L^c : same size as the feature map of last layer
2. ReLU : Only consider features with positive influence

[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Grad-CAM [1]

Step 4 : Visualization: *Upsample* L^c to image size

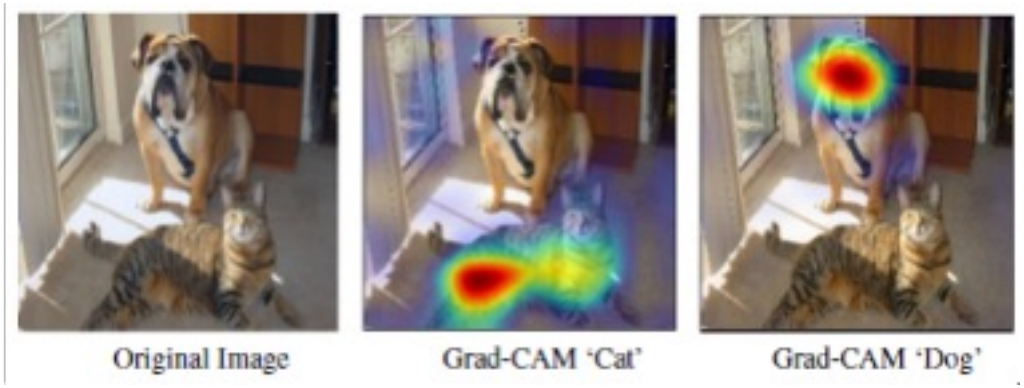


Figure: Grad-CAM results [1]

[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Guided Grad-CAM1 [1]

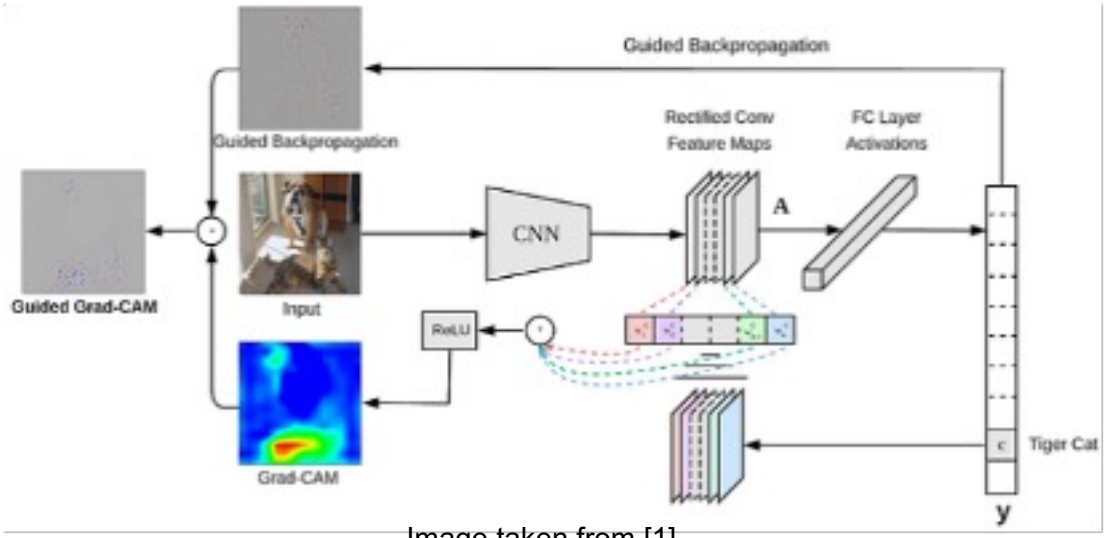
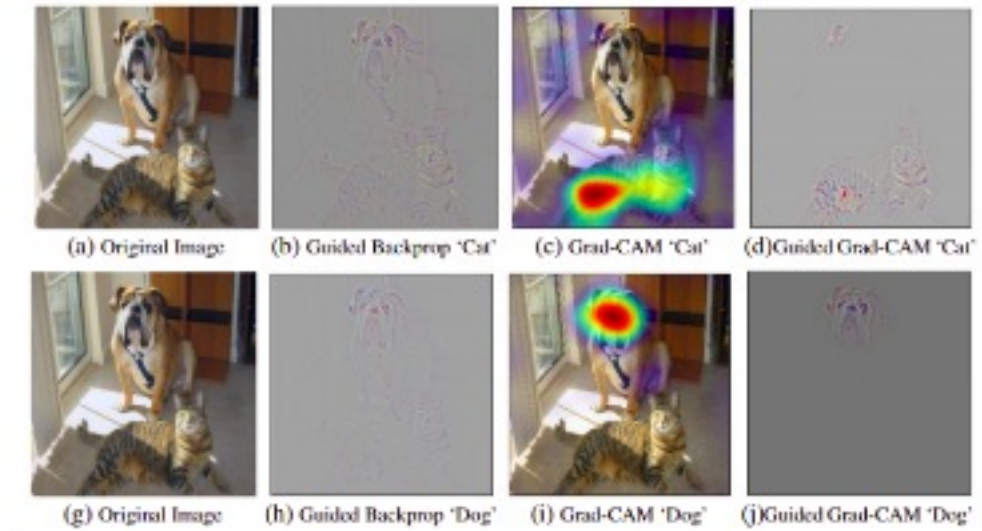


Image taken from [1]

- Grad-CAM lacks the ability to highlight fine-grained details like pixel-space gradient visualization methods.
- In order to combine the best aspects of both, the Guided Backpropagation and Grad-CAM visualizations are fused via element-wise multiplication

[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Guided Grad-CAM1[1]



- It is able to highlight the fine-grained class specific details.

[1] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Methods based on Structure of the Network

- These methods uses the architecture of the network to explain the output
- Starting from the output neuron, these methods use different redistribution rules to propagate the prediction to the input layer to obtain the relevance score maps.
- Unlike the previously seen methods they DO NOT compute gradients in the network.

Layer-Wise Relevance Propagation (LRP) [1]

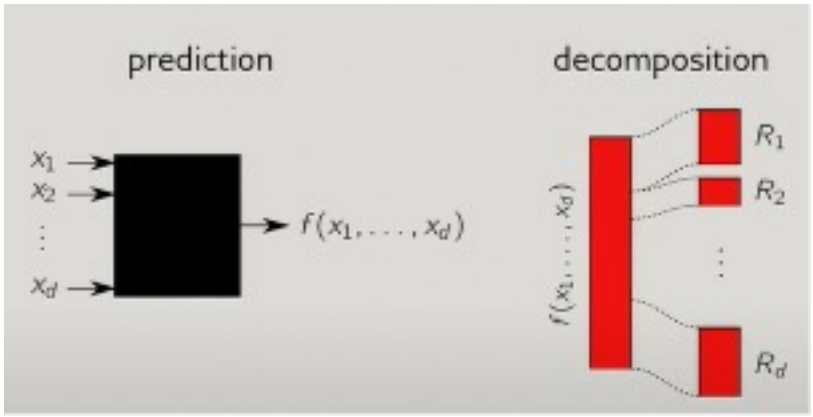


Image taken from the slides of G.Montovan

- LRP explains the decision of the network for a particular image by redistributing the classification score S for a class **c** backwards through the network.
- Instead of gradient calculations it uses the activation of the output neuron as the relevance score
- Starting with the activation of the output neuron for a class **c**, LRP redistributes this score backwards till the input layer-by-layer

[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Layer-Wise Relevance Propagation (LRP) [1]

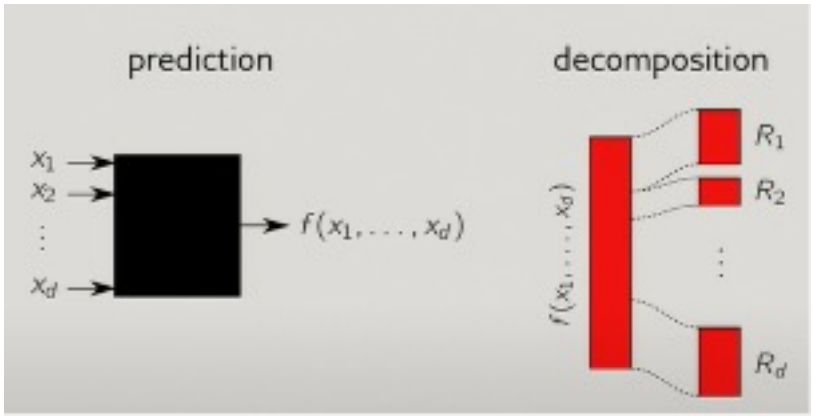


Image taken from the slides of G.Montovan

- Prediction: $f(x_1, x_2, .. x_d)$
  decomposed as Relevance scores
- ***Relevance Conservation:***

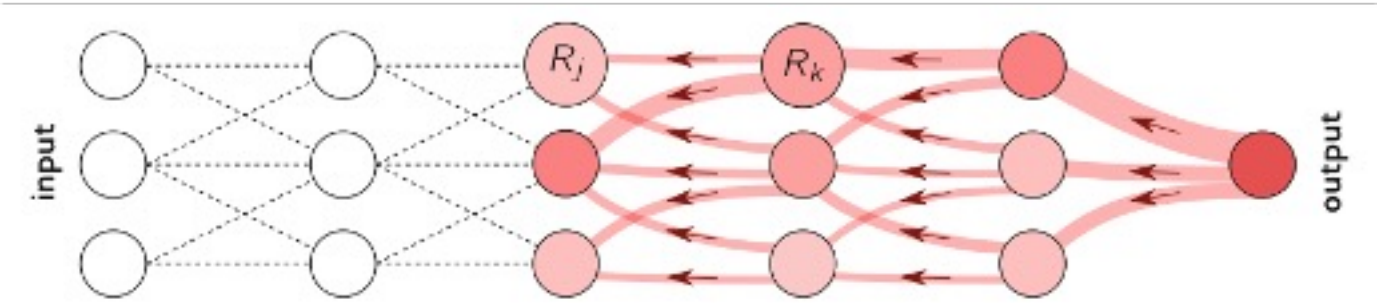
$$f(x_1, x_2, .. x_d) = \sum_{i=1}^{d} R_i$$

- Redistribution:
  - Begin at output layer
  - Propagate layer-layer till Input

Ri are the relevanc escores of each pixel

[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Layer-Wise Relevance Propagation (LRP) [1]



$$R_j = \sum_k \frac{a_j w_{jk}}{\sum_{0,j} a_j w_{jk}} R_k$$

LRP-0 Rule

- a_j: Activation of neuron
- w_{jk} : Weight of the connection between neuron j & k
- k: neurons on the top layer
- j: Neuron on the previous layer

Considering two neurons in two consecutive layers of the network, the LRP-0 rule specifies the simplest LRP of redistributing the score from one layer to the other. The relevance of neuron j is determined by its activation value weighted by the Neurons it is connected to from the previous layer. The term in the denominator is the normalization to ensure relevance conservation

[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Layer-Wise Relevance Propagation (LRP) [1]

This LRP Rule works only with networks that use the ReLU as non-linearity.

At the beginning the relevance of the output neuron is taken before the **Softmax**

$$R_j = \sum_k \frac{a_j w_{jk}}{\epsilon + \sum_{0,j} a_j w_{jk}} R_k$$

Epsilon Rule

- The weights of the connection could be negative => The sum in the denominator could become 0
- ϵ is added for numerical stability

$$R_j = \sum_k \frac{a_j \cdot (w_{jk} + \gamma w_{jk}^+)}{\sum_{0,j} a_j \cdot (w_{jk} + \gamma w_{jk}^+)} R_k$$

Gamma Rule

- (f)+ = max(0,f)
- Gives higher relevance to positive weight connections
- $\boldsymbol{\gamma}$: Controls the influence of positive contributions
- Delivers more stable explanations

[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Layer-Wise Relevance Propagation (LRP) [1]

This LRP Rule works only with networks that use the ReLU as non-linearity.

At the beginning the relevance of the output neuron is taken before the **Softmax**

$$R_i^c = \sum_j \left(\alpha \frac{(a_i w_{ij})^+}{\sum_{0,q}(a_q w_{qj})^+} - \beta \frac{(a_i w_{ij})^-}{\sum_{0,q}(a_q w_{qj})^-} \right) R_j^c$$

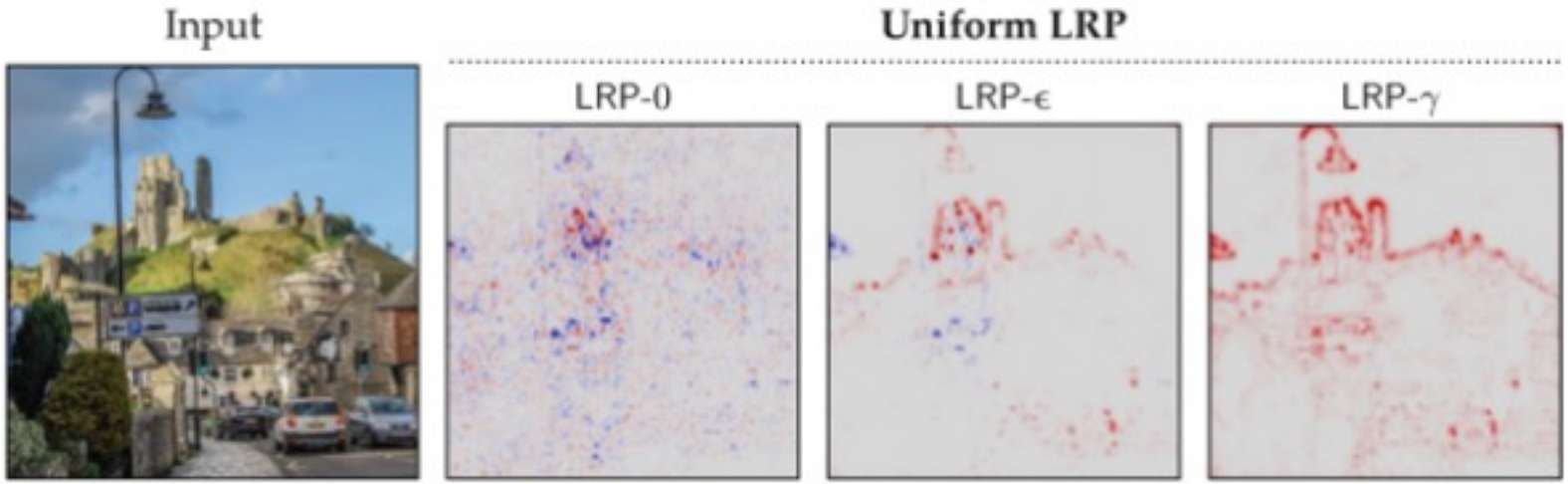
αβ Rule

- The two parameters α and β control the positive and negative contributions to the relevance propagation separately
- Also,

$$(f)^+ = \max(0, f) \qquad (f)^- = \min(0, f) \qquad \alpha = \beta + 1$$

[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Layer-Wise Relevance Propagation (LRP) [1]

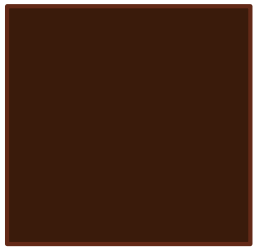


[1]Montavon, G., Binder, A., Lapuschkin, S., Samek, W. and Müller, K.R., 2019. Layer-wise relevance propagation: an overview. In *Explainable AI: interpreting, explaining and visualizing deep learning* (pp. 193-209). Springer, Cham.

Deep Learning Important FeaTures (DeepLIFT) [1]

- Unlike LRP, DeepLIFT establishes the importance of neurons in each layer as the difference of their response to that of a **reference state** (the idea of the reference state is similar to the baseline used in Integrated Gradients)

- The reference state is either a default image/ can be an image chosen based on domain specific knowledge. (The most popular is simply black: RGB (0,0,0)





- DeepLIFT explains the difference in the output produced by the network for an input image and the output of the reference image - based on the *difference between the input image and the chosen reference image*

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.

Deep Learning Important FeaTures (DeepLIFT) [1]

- For the output classification score S_c of the input image x and the output score of reference state as S_c^0 :

- the difference term ΔS_c is defined as $S_c - S_c^0$.

- For the neurons $y_1, \ldots, y_i, \ldots, y_n$ belonging to a layer the relevance is denoted by $R_{\Delta y_i, \Delta S_c}$:

Δy_i denotes the difference between the activations of the neuron y_i for the input image and the reference state. Similar to the LRP method, DeepLIFT has the *Summation to delta* rule where the summation of the relevance of neurons at each layer is constant and equal to ΔS_c as shown in Eq : $\sum_{i=1}^{n} R_{\Delta y_i, \Delta S_c} = \Delta S_c$

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Deep Learning Important FeaTures (DeepLIFT) [1]

In order to explain the *propagation rules*, the authors define a term *multiplier* - $m_{\Delta y \Delta S_c}$. It is a contribution $R_{\Delta y, \Delta S_c}$ of each neuron of the difference in the reference and input image activations Δy to the difference in the output prediction ΔS_c divided by Δy as shown in Eq.

$$m_{\Delta y \Delta S_c} = \frac{R_{\Delta y \Delta S_c}}{\Delta y}$$

The multiplier is a term that is similar to a partial derivative but defined over finite differences.
They also define the *Chain rule for multipliers* similar to the chain rule used with derivatives as shown in Eq. where z_j denotes the neurons in intermediate layers between the neurons y and the output S_c.

$$m_{\Delta y_i \Delta S_c} = \sum_j m_{\Delta y_i \Delta z_j} m_{\Delta z_j \Delta S_c}$$

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Deep Learning Important FeaTures (DeepLIFT) [1]

- Using these terms, DeepLIFT proposes **three rules** that can be applied to a network for different layers to *propagate the relevance from the output to the input layer*.

•**Linear Rule**: The linear rule is applied for the FC and Convolution layers (not applicable for the non-linearity layers). Considering the function

$$z = \sum_i w_i \, y_i + b,$$

where z is the activation of the neuron in the next layer, y_i is the activation to the neuron from the previous layer and w_i is the weight of the connection, the difference with reference state activation is :

$$\Delta z = \sum_i w_i \, \Delta y_i$$

The relevance contribution is then written as

$$R_{\Delta y_i \Delta z} = w_i \Delta y_i.$$

The multiplier in this case is given by

$$m_{\Delta y_i \Delta z} = w_i.$$

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Deep Learning Important FeaTures (DeepLIFT) [1]

• **Rescale Rule**: The Rescale rule is applied to layers with the *non-linearities* like the ReLU.

Consider the neuron z to be the non linear transformation of y as $z = g(y)$. In the case of the ReLU function denoted by $g(y)$.

Considering the summation to delta property, the relevance contribution is $R_{\Delta y \Delta z} = \Delta z$ as there is only one input y. Hence, the multiplier in this case would be

$$m_{\Delta y \Delta z} = \frac{\Delta z}{\Delta y}.$$

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Deep Learning Important FeaTures (DeepLIFT) [1]

• **RevealCancel rule:** The **RevealCancel rule** is applied to **nonlinear operations** (like ReLU, max-pooling, etc.) to **separately track positive and negative contributions** of the inputs **before** they get canceled out by the non-linearity.

This is particularly important in cases where a **non-linearity hides or suppresses part of the signal**. For example, a ReLU will output 0 for all negative values — but DeepLIFT wants to understand whether the **input had a strong negative effect** that was **just masked** by the ReLU.

[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Deep Learning Important FeaTures (DeepLIFT) [1]

The RevealCancel rule defines:

$$R_{\Delta y \Delta S_c}{}^{+} = f(y^{+}) - f(0), R_{\Delta y \Delta S_c}{}^{-} = f(y^{+} + y^{-}) - f(y^{+})$$

Where:

$y^{+}$is the positive part of the input difference Δy

$y^{-}$is the negative part

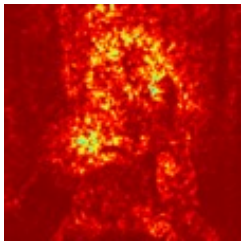
$f(\cdot)$is the nonlinearity (e.g., ReLU)

This gives a **positive contribution** R^{+} and **negative contribution** R^{-}, rather than a single mixed one.

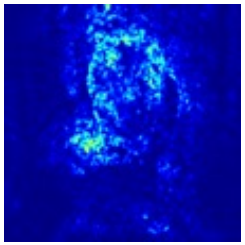
Example ImageNet, ResNet50, class 243 « bull_mastif » - Dog, 224x224



Original Image



Negative Contribution for Dog class

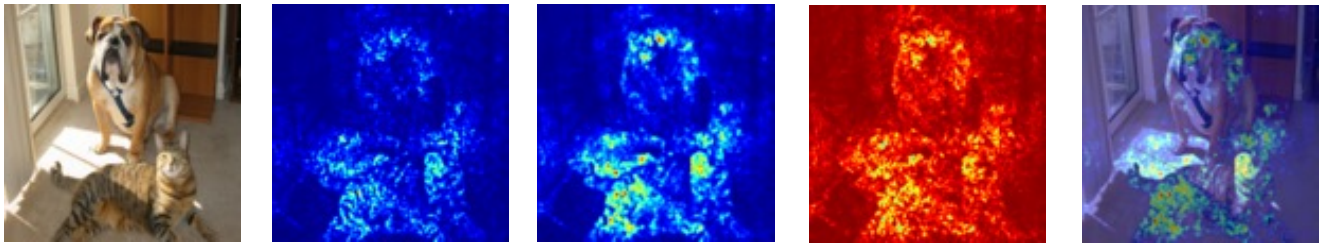


Positive Contribution for Dog class

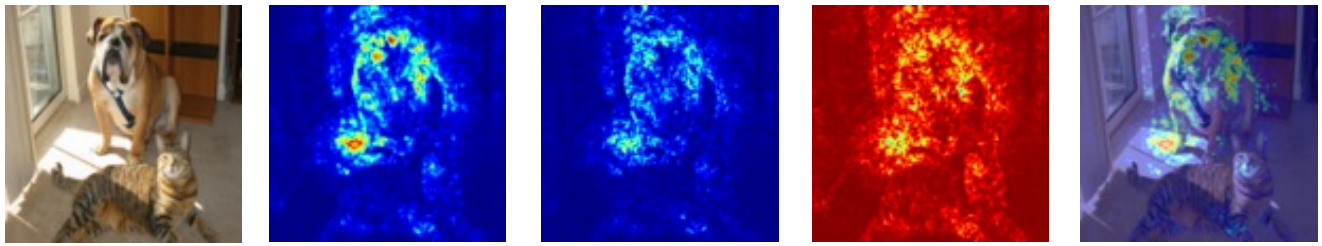
[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.
[2]Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2021. Review of white box methods for explanations of convolutional neural networks in image classification tasks. Journal of Electronic Imaging, 30(5), pp.050901-050901.

Examples : ImageNet, ResNet50 classifier

282 - tiger_cat
(prob= 0.0162)



243 - bull_mastiff
(prob= 0.2764)



Original | Overall importance map | Positive contibutions | Negative contibutions | Superimposed overall

Evaluation of DeepLift [1]

Evaluation : pixels are erased up to 20% accordingly to the importance scores to convert one class to another.

Metric : log-odds score

The **log-odds score** is a **measure of confidence** or **strength** in probabilistic models, especially in **binary classification**. It is the logarithm of the ratio of the **probability of an event occurring** to the **probability of it not occurring**.

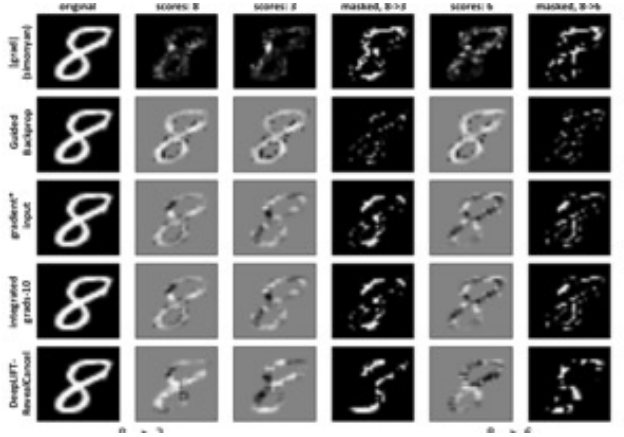
Given a probability $p \in (0, 1)$, the **log-odds** is defined as:

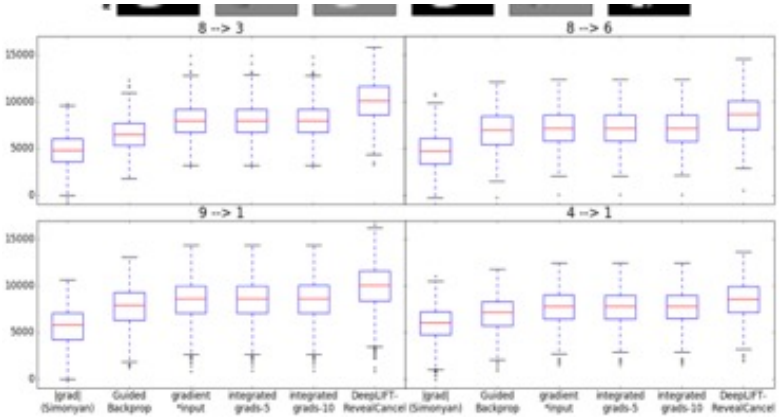
$$\text{log-odds}(p) = \log\left(\frac{p}{1-p}\right)$$

p= probability of a positive class

$\frac{p}{1-p}$= **odds** of the event happening

Mean and std per image in box diagram ->





[1]Shrikumar, A., Greenside, P. and Kundaje, A., 2017, July. Learning important features through propagating activation differences. In International conference on machine learning (pp. 3145-3153). PMLR.

FEM: Feature based Explanation Method [1]

The core of the method relies in the back-tracing of "strong" features from the last feature-layer (conv layer). It "explains" the Network decisions at the generalization step.

K-Sigma Thresholding: Convolutional follows normal distribution. So we can apply $\boldsymbol{\mu \pm k\sigma}$ threshold rule to extract rare important features. Values higher than the threshold is kept.

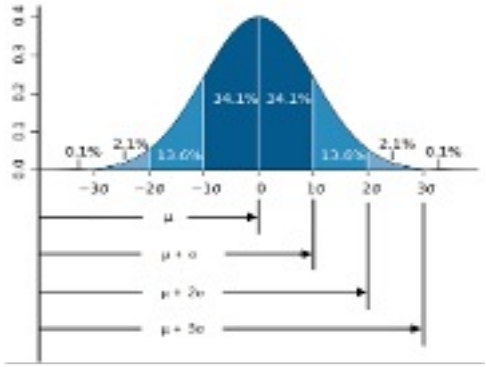


Figure 3: Gaussian Distribution and k-sigma thresholding

Intuition: Conv layers act as feature extractors [2]

Similar to Grad-CAM - using activations from the last conv layer to understand the important features in the input image.

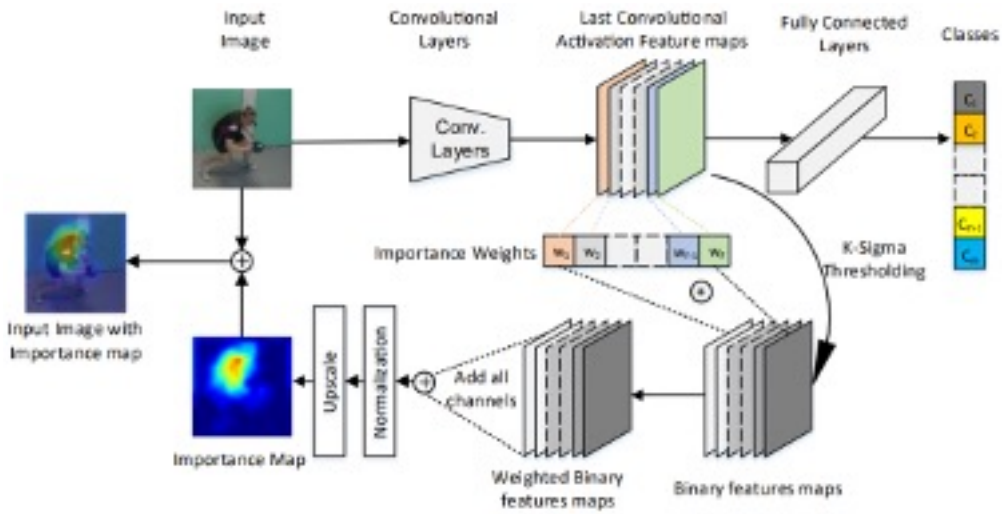
[1] Fuad, K.A.A., Martin, P.E., Giot, R., Bourqui, R., Benois-Pineau, J. and Zemmari, A., 2020, November. Features Understanding in 3D CNNs for Actions Recognition in Video. In *2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA)* (pp. 1-6). IEEE.

FEM: Feature based Explanation Method [1]

Step 1. Forward Pass through the network & store the output of ***desired layer*** & *model output for desired class*.

Step 2. Generate Binary Map from the feature map using K-Sigma Rule.

$$B_k(a_{i,j,k}) = \begin{cases} 1 & \text{if } a_{i,j,k} \geq \mu_k + K * \sigma_k \\ 0 & \text{otherwise} \end{cases}$$



where

k : each feature map from the last conv layer

$\boldsymbol{\mu}$: mean of each feature map

$\boldsymbol{\sigma}$: standard deviation of each feature map

[1] Fuad, K.A.A., Martin, P.E., Giot, R., Bourqui, R., Benois-Pineau, J. and Zemmari, A., 2020, November. Features Understanding in 3D CNNs for Actions Recognition in Video. In *2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA)* (pp. 1-6). IEEE.

[2] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in Advances in neural information processing systems, 2016, pp. 4898–4906.

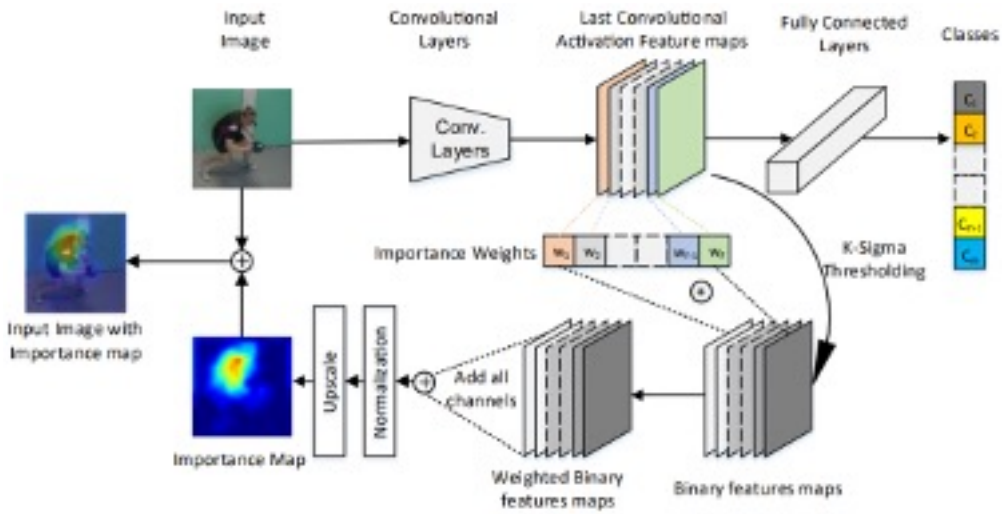
FEM: Feature based Explanation Method [1]

Step 3. **Not all** of the k feature maps are equally important. Thus, to find the importance for each of the **k** channels is set as the mean of the feature map

$$w_k = \mu_k$$

Step 4. Multiply the weights with the Binary map(step 2) and add the channels and generate importance map

Step 5. Normalize the Importance map to [0,1] and upscale to input size to visualize as a heatmap.

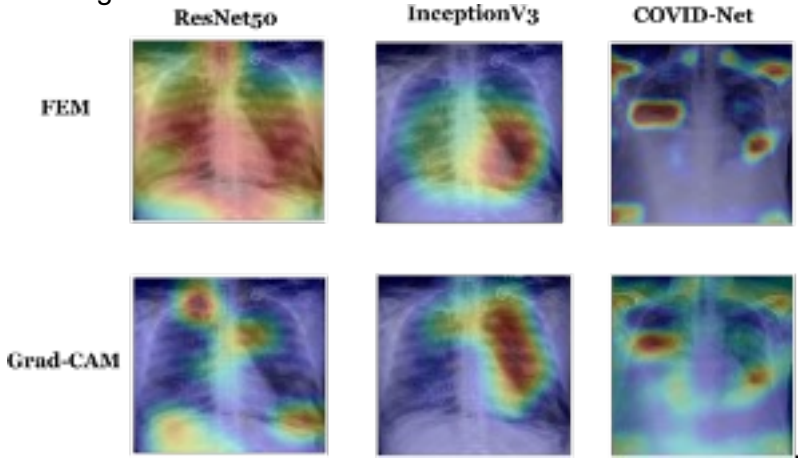


[1] Fuad, K.A.A., Martin, P.E., Giot, R., Bourqui, R., Benois-Pineau, J. and Zemmari, A., 2020, November. Features Understanding in 3D CNNs for Actions Recognition in Video. In *2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA)* (pp. 1-6). IEEE.

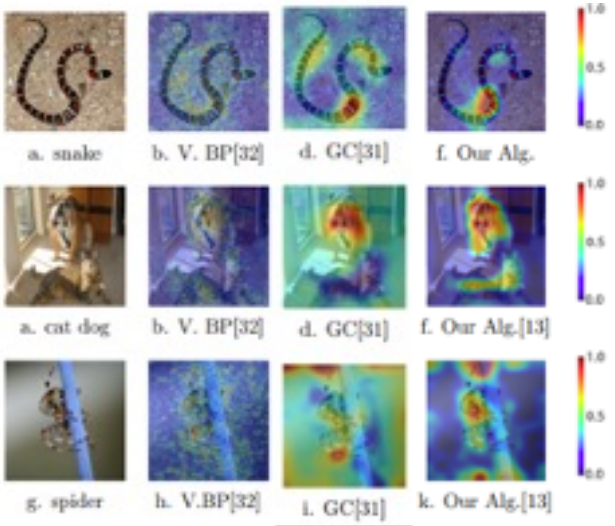
[2] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in Advances in neural information processing systems, 2016, pp. 4898–4906.

FEM: Feature based Explanation Method [1]

- FEM does not need to calculate gradients from the output neuron
- It is a much faster and simpler method based only on the features extracted by the network.
- It is not affected by the classifier head of the network but only the Feature extractor and can be applied to both 2D and 3D images or videos



Results from [2]



Results from [1]

[1] Fuad, K.A.A., Martin, P.E., Giot, R., Bourqui, R., Benois-Pineau, J. and Zemmari, A., 2020, November. Features Understanding in 3D CNNs for Actions Recognition in Video. In *2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA)* (pp. 1-6). IEEE.

[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2023. A feature understanding method for explanation of image classification by convolutional neural networks. In Explainable Deep Learning AI (pp. 79-96). Academic Press.

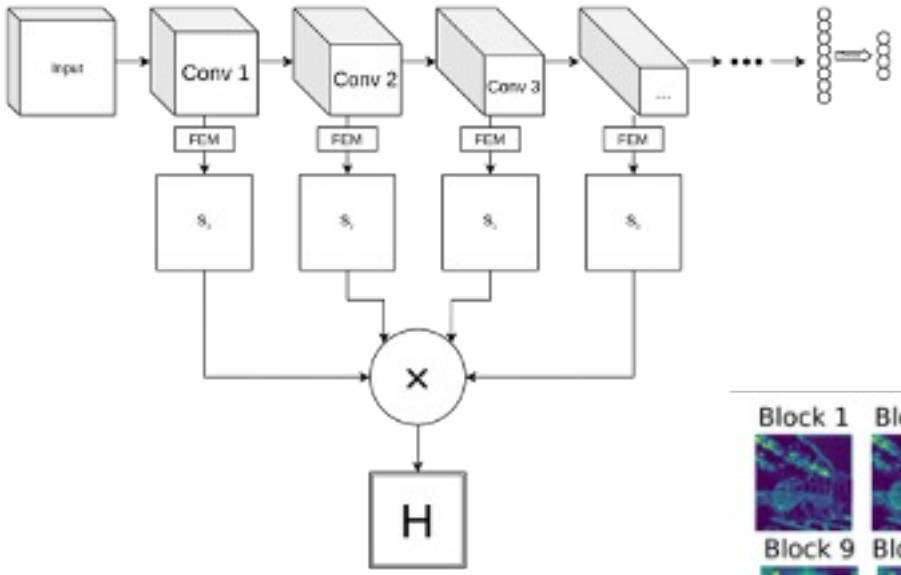
FEM: Feature based Explanation Method [1]

Software to Test and Understand FEM:

https://github.com/labribkb/fem/blob/main/FEM.ipynb

[1] Fuad, K.A.A., Martin, P.E., Giot, R., Bourqui, R., Benois-Pineau, J. and Zemmari, A., 2020, November. Features Understanding in 3D CNNs for Actions Recognition in Video. In *2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA)* (pp. 1-6). IEEE.
[2] Ayyar, M.P., Benois-Pineau, J. and Zemmari, A., 2023. A feature understanding method for explanation of image classification by convolutional neural networks. In Explainable Deep Learning AI (pp. 79-96). Academic Press.

MLFEM: Multi Layered Features Explanation Method [1]



- FEM uses activations only from the **last conv** layer
- Each layer of a CNN embeds information at a different scale, thus computing FEM at several layers and fusing them: **MLFEM**

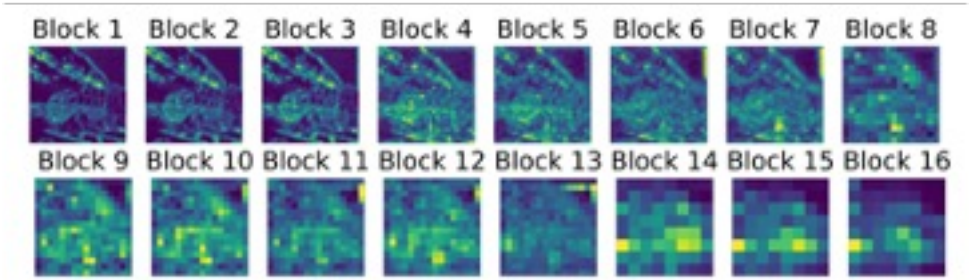


Fig. 1: FEM applied on every convolutional block of a typical ResNet50 architecture. Resolution is higher for the first layers.

[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]

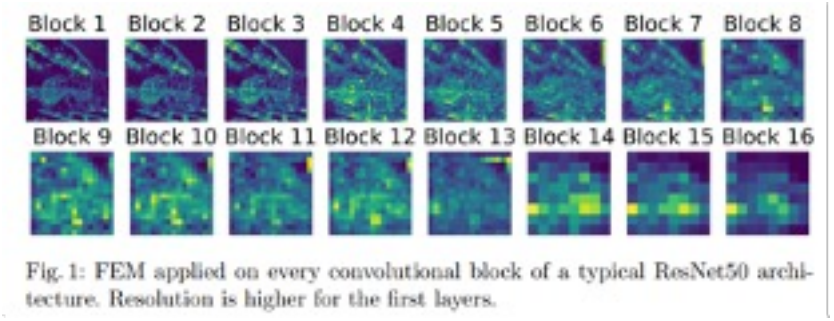


Fig. 1: FEM applied on every convolutional block of a typical ResNet50 architecture. Resolution is higher for the first layers.

Interesting Question: **How to fuse these different maps**

Algebraic Fusion:

- **Add** $f(x,y) = \alpha x + (1-\alpha)y \quad \alpha \in [0,1]$
- **Top** $f(x,y) = \alpha x + (1-\alpha)y'$ $\quad \alpha \in [0,1]$

$$y' = y \left\{ \begin{array}{l} 1 \text{ if } y > \mu_k \\ 0 \text{ otherwise} \end{array} \right\}$$

- **Max** $f(x,y) = \max(x,y)$

[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]

Fusion by learning: “Net”

- Train a small CNN with the input as the feature importance maps from all the CNN layers that we want to explain and the ground-truth as the Gaze Fixation Density Map (GFDM).
- GFDM - expresses human perception of the visual content - these are collected from a number of observers recorded for each image by an eye-tracking device

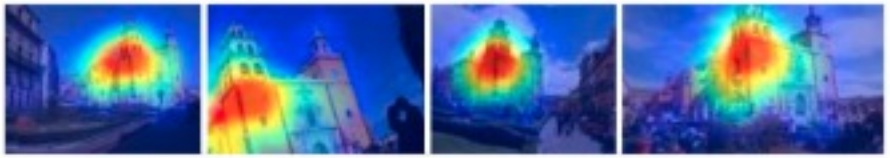
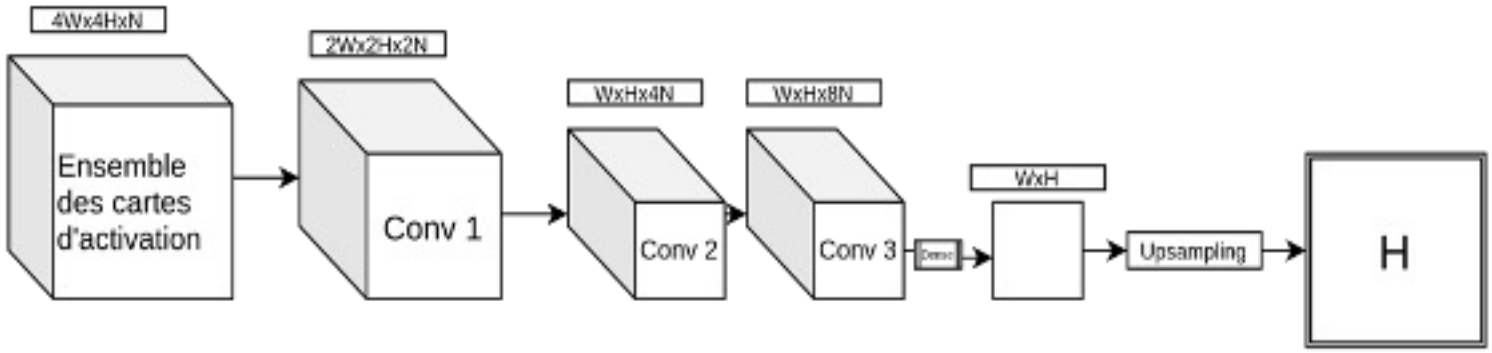


Fig. 2: Example of GFDMs on Mexculture database[10]

- The network *m* is trained using Euclidean loss - the mean squared error loss between the ground truth GFDM and the output of the *m* for the set of intermediate importance feature maps

[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]

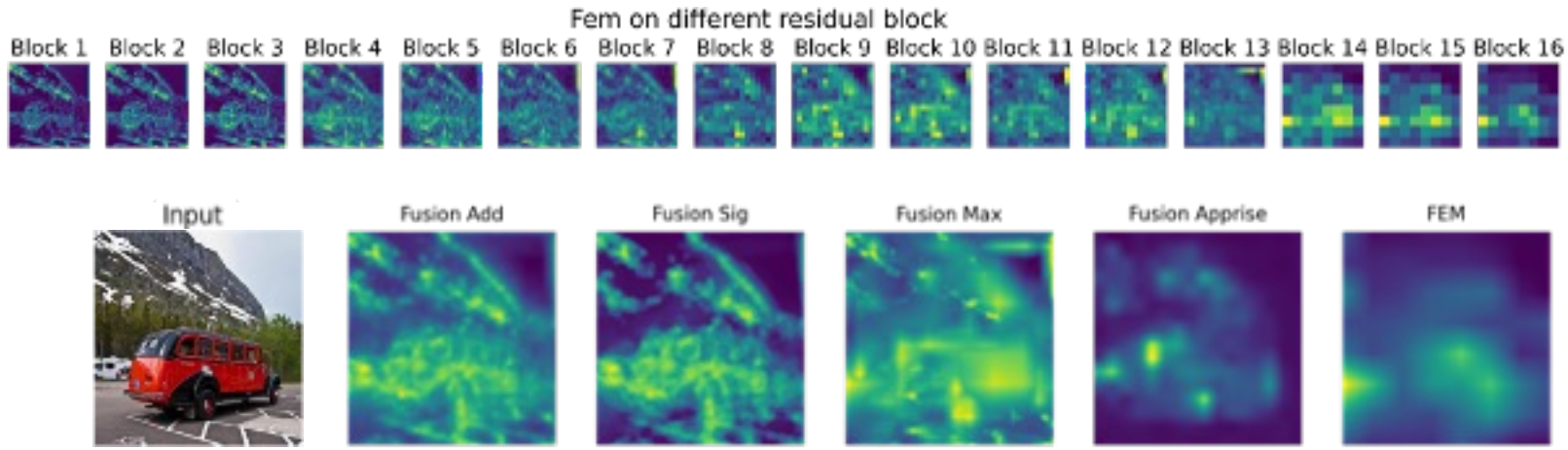


- **Regression function** $loss(y, \hat{y}) = \sum_i (N(\hat{y}_i) - y_i)^2$

Gaze Fixation Density Map $y_i = G(\nu, \eta)$

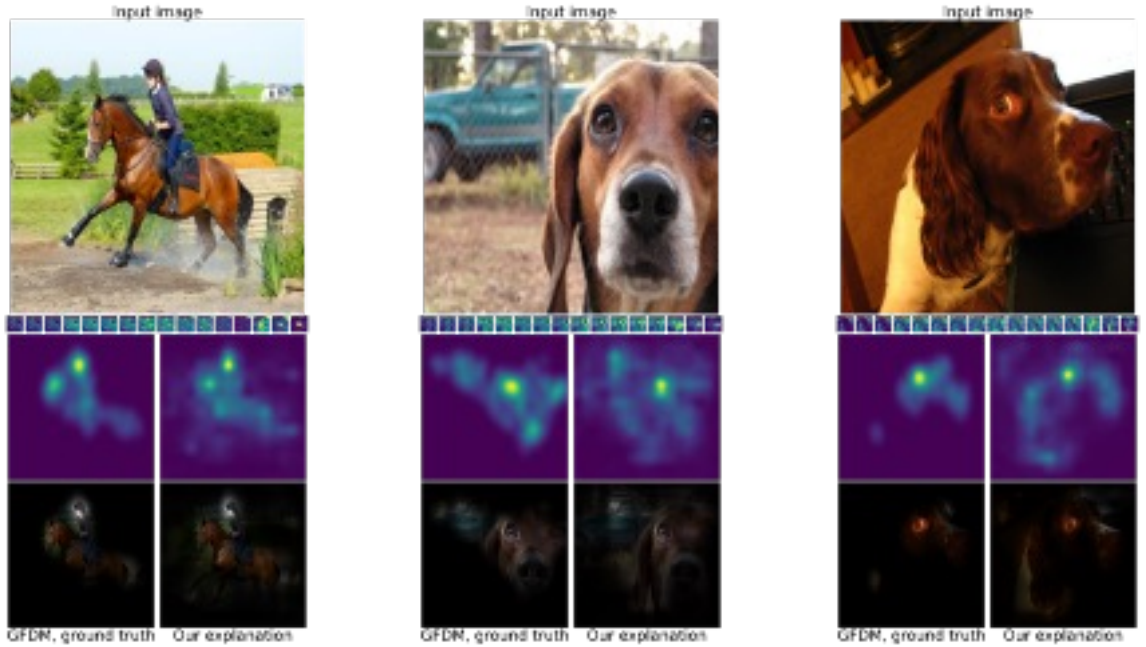
[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]



[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]



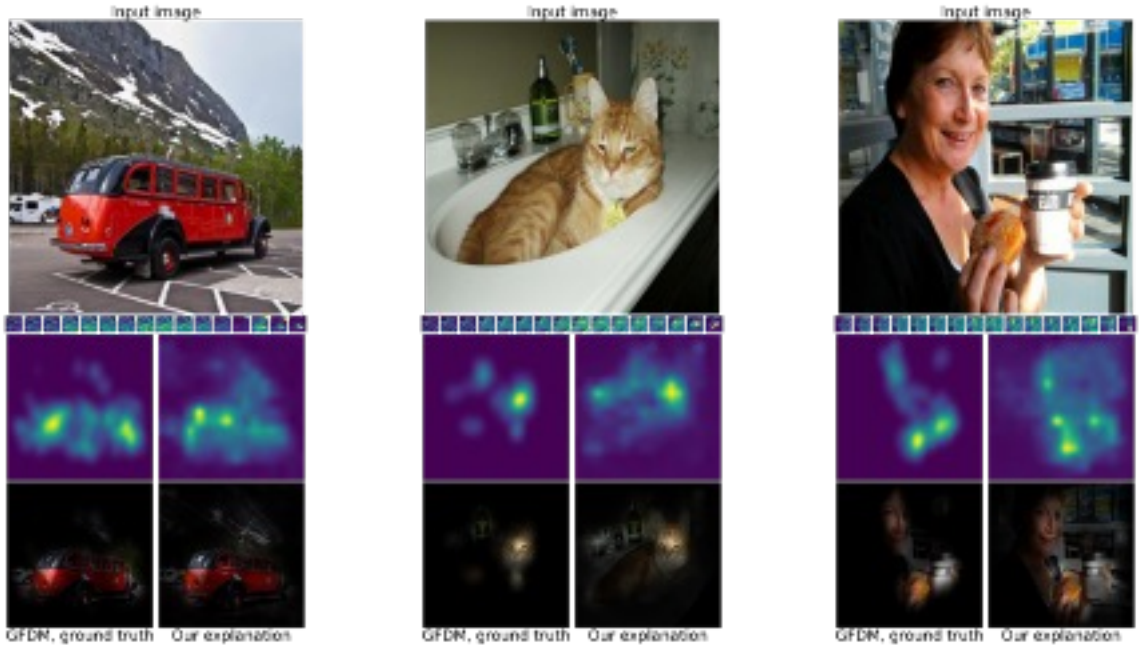
[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]



[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.

MLFEM: Multi Layered Features Explanation Method [1]



[1] Bourroux, L., Benois-Pineau, J., Bourqui, R. and Giot, R., 2022, June. Multi layered feature explanation method for convolutional neural networks. In International Conference on Pattern Recognition and Artificial Intelligence (pp. 603-614). Cham: Springer International Publishing.